

```

/*
=====
OVERHEAD TANK WATER QUALITY & SUPPLY MONITOR
SIH 2026 - LEVEL 2 MODIFIED VERSION
=====

CONTROLLER:
ESP32 DevKit V1

HARDWARE / SIMULATION:
HC-SR04      -> Tank Level
Potentiometer -> Water Quality / TDS Simulation
Potentiometer -> Sump Level Simulation
Relay        -> Pump Control
LED          -> Water Quality Alert
Buzzer       -> Water Quality Alert

=====
ESP32 PIN MAPPING
=====

HC-SR04 TRIG      -> GPIO 5
HC-SR04 ECHO      -> GPIO 18

Quality Sensor    -> GPIO 34
Sump Sensor       -> GPIO 35

Pump Relay        -> GPIO 26
Quality LED       -> GPIO 27
Buzzer            -> GPIO 25
=====
*/

// =====
// 1. PIN DEFINITIONS
// =====
const int TRIG_PIN = 5;
const int ECHO_PIN = 18;

const int QUALITY_PIN = 34;
const int SUMP_PIN = 35;

const int PUMP_RELAY_PIN = 26;
const int QUALITY_LED_PIN = 27;
const int BUZZER_PIN = 25;

// =====
// 2. TANK CONFIGURATION
// =====
const float TANK_CAPACITY_LITRES = 1000.0;
const float EMPTY_DISTANCE_CM = 125.0;
const float FULL_DISTANCE_CM = 25.0;
const float EFFECTIVE_TANK_HEIGHT_CM = EMPTY_DISTANCE_CM - FULL_DISTANCE_CM;

// =====
// 3. PUMP CONTROL
// =====
const float PUMP_START_LEVEL = 30.0;
const float PUMP_STOP_LEVEL = 90.0;

```

```

// =====
// 4. SUMP PROTECTION
// =====
const float SUMP_EMPTY_THRESHOLD = 20.0;

// =====
// 5. WATER QUALITY CONFIGURATION
// =====
const int QUALITY_MIN = 400;
const int QUALITY_MAX = 600;
const int QUALITY_NORMAL_CENTER = (QUALITY_MIN + QUALITY_MAX) / 2;

// =====
// 6. QUALITY MOVING AVERAGE
// =====
const int MOVING_AVERAGE_SIZE = 5;
int qualitySamples[MOVING_AVERAGE_SIZE];
int qualitySampleIndex = 0;
long qualitySampleTotal = 0;
int numberOfQualitySamples = 0;
int smoothedQuality = 0;

// =====
// 7. QUALITY PERSISTENCE
// =====
const int REQUIRED_ABNORMAL_SAMPLES = 5;
int abnormalSampleCount = 0;
bool qualityAlertActive = false;

// =====
// 8. PUMP STATE
// =====
bool pumpState = false;

// =====
// 9. SENSOR FAULT STATE
// =====
bool levelSensorFault = false;

// =====
// 10. TANKER DETECTION
// =====
float previousTankLevel = -1.0;
const float TANKER_RISE_THRESHOLD = 1.0;

// =====
// 11. SAMPLING INTERVAL
// =====
const unsigned long SAMPLE_INTERVAL = 1000;
unsigned long lastSampleTime = 0;

// =====
// SETUP
// =====
void setup() {
  Serial.begin(115200);

  pinMode(TRIG_PIN, OUTPUT);
  pinMode(ECHO_PIN, INPUT);

```

```

pinMode(PUMP_RELAY_PIN, OUTPUT);
pinMode(QUALITY_LED_PIN, OUTPUT);
pinMode(BUZZER_PIN, OUTPUT);

digitalWrite(PUMP_RELAY_PIN, LOW);
digitalWrite(QUALITY_LED_PIN, LOW);
digitalWrite(BUZZER_PIN, LOW);

for (int i = 0; i < MOVING_AVERAGE_SIZE; i++) {
    qualitySamples[i] = 0;
}

Serial.println();
Serial.println("=====");
Serial.println(" OVERHEAD TANK WATER QUALITY & SUPPLY MONITOR ");
Serial.println("=====");
Serial.println("ESP32 SYSTEM STARTED");
Serial.println();
Serial.println("System is ready.");
Serial.println("=====");
}

// =====
// MAIN LOOP
// =====
void loop() {
    unsigned long currentTime = millis();

    if (currentTime - lastSampleTime < SAMPLE_INTERVAL) {
        return;
    }
    lastSampleTime = currentTime;

    Serial.println();
    Serial.println("-----");

    float tankLevel = readTankLevel();

    if (tankLevel < 0) {
        handleLevelSensorFault();
        readAndMonitorQuality();
        Serial.println("-----");
        return;
    }

    levelSensorFault = false;
    float tankLitres = calculateLitres(tankLevel);
    float sumpLevel = readSumpLevel();

    Serial.println("TANK INFORMATION");
    Serial.print("Tank Level: ");
    Serial.print(tankLevel, 1);
    Serial.println("%");
    Serial.print("Tank Volume: ");
    Serial.print(tankLitres, 1);
    Serial.println(" L");

    Serial.println();
    Serial.print("Sump Level: ");

```

```

Serial.print(sumpLevel, 1);
Serial.println("%");

controlPump(tankLevel, sumpLevel);
readAndMonitorQuality();
detectTankerFill(tankLevel);

previousTankLevel = tankLevel;
Serial.println("-----");
}

// =====
// FUNCTION: READ TANK LEVEL
// =====
float readTankLevel() {
    digitalWrite(TRIG_PIN, LOW);
    delayMicroseconds(2);
    digitalWrite(TRIG_PIN, HIGH);
    delayMicroseconds(10);
    digitalWrite(TRIG_PIN, LOW);

    unsigned long duration = pulseIn(ECHO_PIN, HIGH, 30000);

    if (duration == 0) {
        Serial.println();
        Serial.println("!!! ULTRASONIC SENSOR FAULT !!!");
        Serial.println("Reason: No Echo Received");
        return -1;
    }

    float distance = (duration * 0.0343) / 2.0;

    Serial.print("Ultrasonic Distance: ");
    Serial.print(distance, 2);
    Serial.println(" cm");

    if (distance < FULL_DISTANCE_CM || distance > EMPTY_DISTANCE_CM) {
        Serial.println();
        Serial.println("!!! IMPOSSIBLE SENSOR READING !!!");
        Serial.print("Measured Distance: ");
        Serial.print(distance, 2);
        Serial.println(" cm");
        return -1;
    }

    float waterHeight = EMPTY_DISTANCE_CM - distance;
    float levelPercentage = (waterHeight / EFFECTIVE_TANK_HEIGHT_CM) * 100.0;

    if (levelPercentage < 0.0 || levelPercentage > 100.0) {
        return -1;
    }

    return levelPercentage;
}

// =====
// FUNCTION: HANDLE LEVEL SENSOR FAULT
// =====
void handleLevelSensorFault() {
    levelSensorFault = true;

```

```

    stopPump();
    Serial.println();
    Serial.println("=====");
    Serial.println("          LEVEL SENSOR FAULT");
    Serial.println("=====");
    Serial.println("Safety Action: PUMP FORCED OFF");
    Serial.println("=====");
}

// =====
// FUNCTION: CALCULATE LITRES
// =====
float calculatelitres(float levelPercentage) {
    return (levelPercentage / 100.0) * TANK_CAPACITY_LITRES;
}

// =====
// FUNCTION: READ SUMP LEVEL
// =====
float readSumpLevel() {
    int rawValue = analogRead(SUMP_PIN);
    float sumpPercentage = ((float)rawValue / 4095.0) * 100.0;
    return constrain(sumpPercentage, 0.0, 100.0);
}

// =====
// FUNCTION: CONTROL PUMP
// =====
void controlPump(float tankLevel, float sumpLevel) {
    if (levelSensorFault) {
        stopPump();
        return;
    }

    if (sumpLevel <= SUMP_EMPTY_THRESHOLD) {
        stopPump();
        Serial.println("DRY-RUN PROTECTION ACTIVE | PUMP: OFF");
        return;
    }

    if (tankLevel <= PUMP_START_LEVEL) {
        startPump();
        Serial.println("Tank Level <= 30% | PUMP: ON");
    }
    else if (tankLevel >= PUMP_STOP_LEVEL) {
        stopPump();
        Serial.println("Tank Level >= 90% | PUMP: OFF");
    }
}

// =====
// FUNCTION: START PUMP
// =====
void startPump() {
    if (!pumpState) {
        digitalWrite(PUMP_RELAY_PIN, HIGH);
        pumpState = true;
    }
}

```

```

// =====
// FUNCTION: STOP PUMP
// =====
void stopPump() {
    digitalWrite(PUMP_RELAY_PIN, LOW);
    pumpState = false;
}

// =====
// FUNCTION: READ AND MONITOR QUALITY
// =====
void readAndMonitorQuality() {
    int rawQuality = analogRead(QUALITY_PIN);
    smoothedQuality = updateQualityAverage(rawQuality);
    int qualityDeviation = smoothedQuality - QUALITY_NORMAL_CENTER;

    Serial.println();
    Serial.println("WATER QUALITY MONITORING");
    Serial.print("Raw Quality ADC: ");
    Serial.println(rawQuality);
    Serial.print("Smoothed Quality: ");
    Serial.println(smoothedQuality);
    Serial.print("Quality Deviation: ");
    Serial.print(qualityDeviation >= 0 ? "+" : "");
    Serial.print(qualityDeviation);
    Serial.println(" ADC");

    monitorQualityPersistence(smoothedQuality);
}

// =====
// FUNCTION: UPDATE QUALITY MOVING AVERAGE
// =====
int updateQualityAverage(int newValue) {
    qualitySampleTotal += newValue;
    qualitySamples[qualitySampleIndex] = newValue;
    qualitySampleIndex++;

    if (qualitySampleIndex >= MOVING_AVERAGE_SIZE) {
        qualitySampleIndex = 0;
    }

    if (numberOfQualitySamples < MOVING_AVERAGE_SIZE) {
        numberOfQualitySamples++;
    } else {
        qualitySampleTotal = 0;
        for (int i = 0; i < MOVING_AVERAGE_SIZE; i++) {
            qualitySampleTotal += qualitySamples[i];
        }
    }

    return qualitySampleTotal / numberOfQualitySamples;
}

// =====
// FUNCTION: QUALITY PERSISTENCE DETECTION
// =====
void monitorQualityPersistence(int quality) {
    bool abnormal = (quality < QUALITY_MIN || quality > QUALITY_MAX);

```

```

    if (abnormal) {
        abnormalSampleCount++;
        if (abnormalSampleCount >= REQUIRED_ABNORMAL_SAMPLES) {
            if (!qualityAlertActive) {
                qualityAlertActive = true;
                digitalWrite(QUALITY_LED_PIN, HIGH);
                digitalWrite(BUZZER_PIN, HIGH);
                Serial.println("!!! QUALITY SHIFT DETECTED !!!");
            }
        }
    } else {
        abnormalSampleCount = 0;
        if (qualityAlertActive) {
            qualityAlertActive = false;
            digitalWrite(QUALITY_LED_PIN, LOW);
            digitalWrite(BUZZER_PIN, LOW);
            Serial.println("Quality Alert: CLEARED");
        }
    }
}

// =====
// FUNCTION: TANKER FILL DETECTION
// =====
void detectTankerFill(float currentTankLevel) {
    if (previousTankLevel < 0) return;
    float levelRise = currentTankLevel - previousTankLevel;

    if (!pumpState && levelRise >= TANKER_RISE_THRESHOLD) {
        Serial.println("POSSIBLE TANKER FILL DETECTED");
    }
}

```